Token Averaging for Efficient LM Training

Progress report: hypothesis, experiments, evaluation fix, and scaling plan

Vardh

July 2026

Outline

The idea

Standard LM training: one transformer position per token. Cost per token is fixed; more data $\Rightarrow$ proportionally more compute.

Token averaging: average every k consecutive token *embeddings* into one position *before* the transformer.

- $k = 2$: sequence of T raw tokens $\rightarrow$ $T/2$ transformer positions
- Position j = mean of embeddings of tokens $(jk, \ldots, jk+k-1)$
- Training target at position j: the *first token of the next window*, i.e. token $(j+1)k$

Hypothesis

The transformer can model compressed token representations nearly as well as raw ones. Then, at a fixed compute budget, a k-averaged model trains on $\sim k\times$ more raw data (or sees $k\times$ more context) than the baseline — and reaches **lower loss for the same FLOPs**.

Method in one picture

raw tokens	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
embeddings	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8
averaged (k=2)	$\bar{e}_1 = \frac{e_1+e_2}{2}$		$\bar{e}_2 = \frac{e_3+e_4}{2}$		$\bar{e}_3 = \frac{e_5+e_6}{2}$		$\bar{e}_4 = \frac{e_7+e_8}{2}$	
targets	$\rightarrow t_3$		$\rightarrow t_5$		$\rightarrow t_7$		—	

- Transformer runs on $\bar{e}_{1..3}$ (half the positions), standard CE loss
- No new parameters: plain mean pooling between embedding and body
- Same architecture, tokenizer, optimizer, and data as the k=1 baseline

Setup

Models (GPT-NeoX-style, RoPE, SwiGLU, trained from scratch on FineWeb):

	d_{model}	heads	layers	raw tokens	config
50M k=1 / k=2	512	8	8	1B / 2B	seq_len 1024
125M k=1 / k=2	768	12	12	2.5B / 5B	seq_len 1024
250M k=1 / k=2	1024	16	16	5B / 10B	seq_len 1024

Design: same `seq_len` = 1024 *raw* tokens for both; the k=2 transformer therefore sees 512 positions per sequence. Token budgets set so both runs end at $\approx$ equal total training FLOPs.

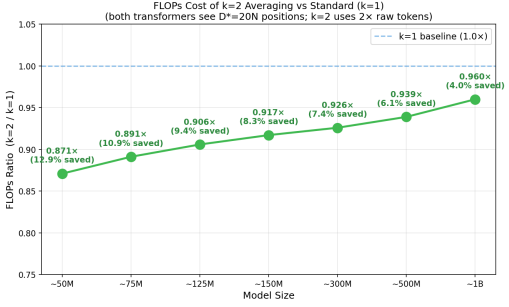
FLOPs accounting (per layer, per transformer position, fwd+bwd):

$$\text{FLOPs} = \frac{\text{tokens}}{\text{seq\_len}} \cdot n_{\text{layers}} \cdot L \cdot 3\,(23d^2 + 4Ld), \qquad L = \text{seq\_len}/k$$

Where the FLOPs saving actually comes from

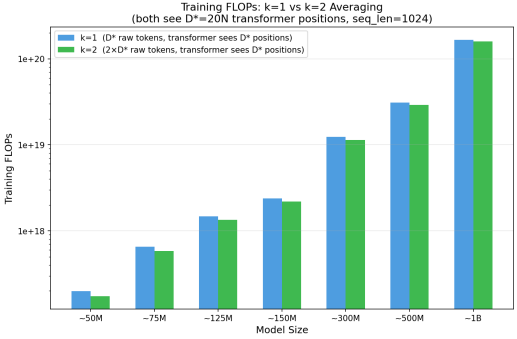
Per *raw* token, k=2 halves the number of positions:

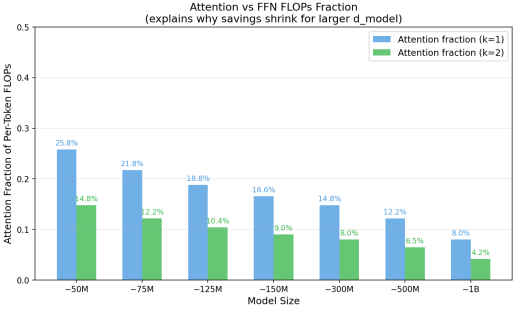
- Linear terms ($23d^2$: QKV, output proj, FFN): 2× tokens × $\frac{1}{2}$ positions = **exactly cancels**
- Attention term ($4Ld$): additionally benefits from $L \to L/2 \Rightarrow$ **half the attention FLOPs survive**



At iso-data-multiple, the raw FLOPs saving is only the attention share — and it shrinks with d_{model}.

FLOPs picture across scales

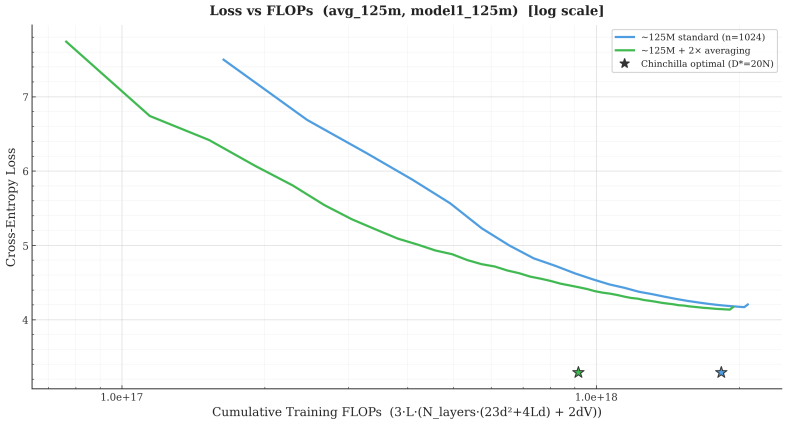




Key reframing

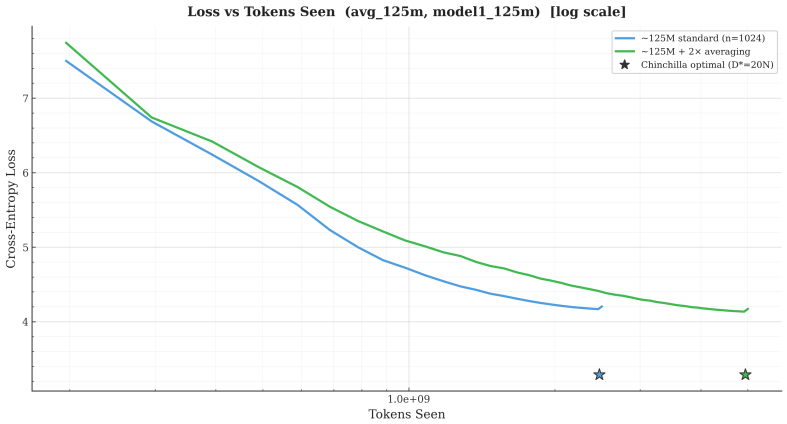
The arithmetic saving (4–13%) is not the story. The story is: k=2 **embeds** 2× **the raw data into the same transformer budget.** The value of the method is data/context throughput per FLOP, not cheaper matmuls.

125M: loss vs training FLOPs (iso-FLOPs endpoints)



k=2 (green) dominates the baseline over the whole trajectory at equal FLOPs.

125M: loss vs raw tokens seen



Per raw token the averaged model learns *slightly less* (green above blue) — but each of its tokens costs half the compute, so it wins on the FLOPs axis.

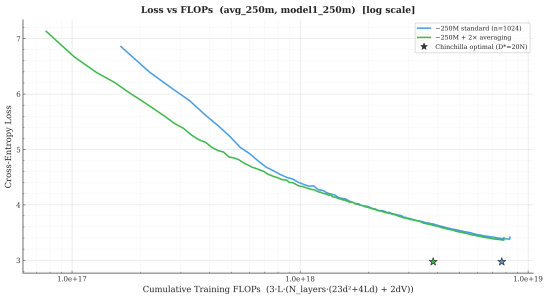
Endpoint numbers (training-log eval)

Run	Raw tokens	Train FLOPs	Final eval	Running eval
125M k=1	2.5B	1.50×10^{18}	4.204	4.168
125M k=2	5.0B	1.36×10^{18}	4.171	4.135
250M k=1	5.0B	6.79×10^{18}	3.414	3.380
250M k=2	10.0B	6.29×10^{18}	3.403	3.362

Two ways to read the same curves:

- **Equal compute, compare loss (vertical):** at the k=2 budget of 1.36×10^{18}, baseline is at 4.185 vs 4.171 — k=2 is better while also having spent 0.91× the FLOPs at its endpoint
- **Equal loss, compare compute (horizontal):** baseline needs its full 1.50×10^{18} to reach 4.204; the k=2 curve crosses 4.204 at $1.05 \times 10^{18} \Rightarrow$ **1.44×** less compute. (Small vertical gaps = large horizontal gaps: loss curves are very flat late in training)
- 50M pair, now using the **matched-protocol** k=2 **rerun** (4.363 vs baseline 4.437, same tokens/update, both tied): **1.58×** saving. The old-protocol k=2 log gave 1.15× for the same comparison — update schedule alone moves the multiplier
- 250M pair (full budgets, random-offset training): **1.32×** saving by the same reading
- Multiplier across scales: 1.58 → 1.44 → 1.32 → noise floor at 500M (next section) — shrinking, not growing; and all pairs share the update-count confound resolved on the next slides

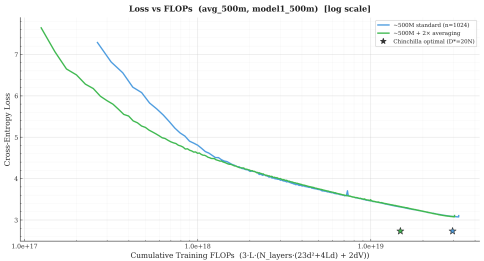
250M pair complete (random-offset training, full budgets)



Run	Raw tokens	Train FLOPs	Final eval	Running eval	Saving
250M k=1	5.0B	6.79×10^{18}	3.414	3.380	—
250M k=2	10.0B	6.29×10^{18} (0.93×)	3.403	3.362	1.32×

Same qualitative picture as 50M/125M: k=2 ends below the baseline at fewer FLOPs. But the equal-loss multiplier keeps shrinking with scale, and at iso-FLOPs the k=2 arm still takes 2× the optimizer updates (both arms ran 32,768 raw tokens/update) — the confound the next slide isolates.

500M pair complete: the decay reaches the noise floor



Run	Raw tokens	Train FLOPs	Full-position eval
500M k=1	10.0B	3.22×10^{19}	3.073
500M k=2	20.0B	3.05×10^{19} (0.946×)	3.071

The sign flips depending on which metric you read. Native loss now slightly *favors the baseline* (−0.003 nats); the full-position metric (comparable across k) still favors averaging (+0.003 nats). Both are an order of magnitude below the 0.070-nat shift the training protocol alone caused at 50M, with zero seed replicates to bound noise at this scale — not a clean win or a clean loss.

FLOPs include the output projection ($2dV$); the 125M/250M numbers on earlier slides predate that fix and are not directly comparable to this table until reconciled.

What the 500M point means: the gap shrinks but holds

On the **full-position eval** (the comparable metric across k), averaging still wins at every scale where we have measured it:

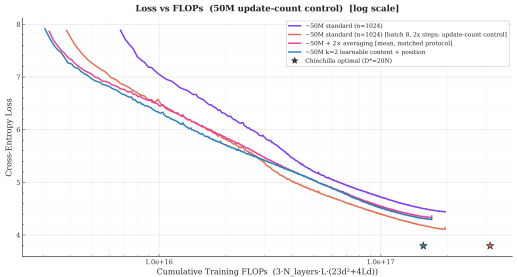
Scale	Native gap (k=1 − k=2)	Full-position gap	Status
50M	0.074	(not yet measured)	
125M	0.033	0.037	k=2 wins
250M	0.011	(not yet measured)	
500M	−0.003	0.003	k=2 still wins

- On the native metric the gap crossed zero at 500M (−0.003), but native loss is not comparable across k — this is the metric we declared invalid for cross-k comparison
- On the full-position metric, averaging still wins at 500M (+0.003 nats), though the margin is tiny and shrinking (0.037 → 0.003 from 125M to 500M)
- The gap is small enough to be within single-seed noise (we have zero replicates at this scale). Seed replicates at 500M would settle whether this is a real residual win or noise
- Missing: full-position eval at 50M and 250M (currently only native gaps; rescore queued)

Reading

The decay trend is real: the advantage shrinks with scale. But on the comparable metric it has *not* crossed zero yet. Whether it reaches zero at 1B or stabilizes at a small positive value is the open question.

Update-count control: the 50M gain does not survive



Run (50M)	Raw tokens	Updates	Train FLOPs	Final eval
k=1 baseline (batch 16)	1B	61k	1.95×10^{17}	4.437
k=2 mean, matched	2B	122k	1.70×10^{17}	4.363
k=2 learnable+pos	2B	122k	1.70×10^{17}	4.326
k=1 **control (batch 8)**	1B	**122k**	1.95×10^{17}	**4.142**

The control is the baseline with half the batch: same 1B raw tokens and FLOPs, but the *same 122k updates and the same 8,192 transformer positions per update as the k=2 arms*. Only the input resolution differs.

Update-count control: readings

- The control beats every k=2 variant: 4.142 vs 4.326 for the best pooling rule. It reaches k=2 mean's final loss (4.363) at 1.06×10^{17} FLOPs — 0.62× of what k=2 spent — and learnable+pos's 4.326 at 59% of its own budget
- Update schedule explains more than the whole 50M headline: batch 16 → 8 at identical data and FLOPs moves the baseline by −0.295 nats; the k=2 edge over the batch-16 baseline was only −0.074
- At 61k updates (same count as the batch-16 baseline, but only 0.5B tokens and half the FLOPs), the control has *already* matched the baseline's final loss (4.420 vs 4.443) — at this scale, loss tracks update count far more than data seen
- Metric caveat: these are native losses (cross-k), but at 125M the native and full-position readings differed by ∼0.005 nats — a 0.22-nat gap will not flip

Implication

At 50M, with updates and positions matched, **raw tokens beat averaged tokens**. The iso-FLOPs training-efficiency claim now rests entirely on whether this control repeats at 125M/250M — run it there before any 500M/1B spend. (Open question in averaging's favor: does k=2 also improve with batch 8? Both arms deserve the tuned setting.)

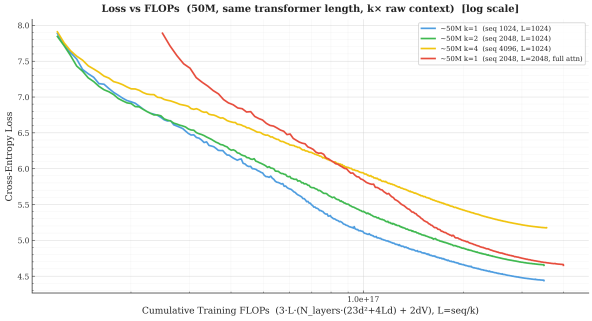
New experiment: is it a data effect or a context effect?

Two ways to deploy k-averaging at a fixed FLOP budget:

	seq_len (raw)	Transformer len	Raw context	Tokens trained
k=1 baseline	1024	1024	1024	1B
Config A: k=2, same seq	1024	512	1024	2B
Config B: k=2, same L	2048	1024	2048	2B
Config B: k=4, same L	4096	1024	4096	4B
k=1 full attention	2048	2048	2048	1B

- Config A pockets the compute saving (shorter transformer sequences)
- Config B spends it on *extended raw context* at the same transformer length — "double context for free"
- Per raw token, Config B costs exactly $1/k$ of baseline $\Rightarrow$ iso-FLOPs at $k\times$ tokens, by construction
- k=1 full attention: the *traditional* way to get 2048 context — costs 1.26$\times$ baseline per token (L=2048 attention)

Result: extended context loses at ∼iso-FLOPs (50M)



Run	Raw tokens	FLOPs	Final eval	vs baseline
k=1, seq 1024	1.00B	1.95×10^{17}	4.437	—
k=2, seq 2048	2.00B	1.95×10^{17}	4.650	+0.213
k=1, seq 2048 (full attn)	1.00B	2.45×10^{17}	4.651	+0.214
k=4, seq 4096	4.07B	1.99×10^{17}	5.174	+0.739

Context-for-context: averaging beats full attention

Both 2048-raw-context arms land at (almost eerily) **identical final loss** — at very different cost:

2048-context model	Final eval	FLOPs	KV cache	Data seen
k=2 averaging (L=1024)	4.6503	1.95×10^{17}	$\frac{1}{2}$	2B
k=1 full attn (L=2048)	4.6505	2.45×10^{17}	1	1B

- Averaging reaches full attention's final loss with **1.26**× less compute; at averaging's budget, full attention is still at 4.727
- Plus half the KV cache and attention cost at inference for the same raw context

Two takeaways, one experiment

(1) If you want longer raw context, averaging is a strictly cheaper way to get it than widening the attention window. **(2)** But on packed FineWeb at 50M, context beyond 1024 hurts *by either method* (+0.21 nats) — the failure is the data/scale, not the averaging.

The decomposition: cheaper tokens, not more context

Same model, same k=2, same 2B raw tokens — only where the compute goes differs:

	Final eval loss	FLOPs vs baseline	Outcome
Config A (k=2, seq 1024)	4.363	0.87×	wins (1.58× saving)
Config B (k=2, seq 2048)	4.650	1.00×	loses; baseline is 1.72× cheaper

Config A value is the matched-protocol rerun (16k tokens/update, 122k steps); Config B ran at 32k tokens/update (61k steps, same update count as the baseline).

Interpretation

The benefit of token averaging is **data-per-FLOP, not context**. Extended raw context does not pay for itself at this scale — *by any method*:

- The k=1 full-attention control at 2048 context is equally bad (+0.21 nats) at 1.26× the cost ⇒ the failure is the data/scale, not averaging
- FineWeb docs are mostly <1024 tokens; packing crosses document boundaries unmasked ⇒ context beyond 1024 is largely other documents' text
- Degradation compounds with k (+0.21 → +0.74 nats), consistent with patch-size findings in prior work

Caveat before generalizing: needs a loss-vs-position check and a long-document dataset (PG19 / code) to rule context in or out at scale.

Detours and bugs (and what they taught us)

1. RoPE bug in early runs — invalidated the first tied-embedding and several old k-sweep results. Old logs quarantined; only post-fix runs are used.
2. 50M k=1 baseline instability — the original run ended ∼1 nat too high after an early loss explosion. Rerun completed: clean baseline now ends at 4.437, consistent with the 125M scaling pattern. (All runs show a large transient init loss that self-recovers by step ∼2000 — an init artifact, clipped in plots.)
3. The "seq_len bug" that became the design — k=2 was accidentally trained with the same raw `seq_len`=1024 as k=1 (not 2048). This turned out to be the *cleaner* control: both models condition on identical raw context, isolating the data-per-FLOP effect from the context effect.
4. Evaluation comparability — the most serious issue; next section.

The problem: our losses were not measuring the same task

- k=1 loss: average CE over predictions of **every** token
- k=2 loss: average CE over **every 2nd token only** (first token of each next window); odd positions get *no probability at all*

A language model's defining quantity is the full-sequence likelihood

$$\sum_i \log p(t_i \mid t_{<i})$$

— for the averaged model, half the terms are missing. It also cannot generate token-by-token as trained.

Reviewer's one-liner

"Your model matches the baseline on the half of the task it attempts."

Two candidate fixes

Approach A — match the prediction budget (ours, first attempt): evaluate $k{=}2$ on $2\times$ the eval batches so the *count* of predicted tokens matches $k{=}1$.

- Issue: eval loss is a per-prediction *mean*; more batches only shrink the error bar of the same quantity. Coverage stays even-positions-only; still no sequence likelihood. Count matches, task doesn't.

Approach B — offset-ensemble evaluation (adopted): run the model k times per sequence, shifting the window grid by $o = 0, \ldots, k{-}1$ tokens.

- Offset 0 predicts tokens $2, 4, 6, \ldots$; offset 1 predicts $3, 5, 7, \ldots$
- Together: every position predicted exactly once, each from its full compressed prefix $\Rightarrow$ true full-sequence NLL
- Cost: k passes at $1/k$ length $\approx$ one baseline forward pass
- Baseline additionally scored *restricted to the identical position set* $\Rightarrow$ airtight comparison

Full-position evaluation: results (125M, final checkpoints)

Same eval sequences, same target positions, 3,409,119 predictions each:

Model	Mean NLL (nats/token)	PPL	Coverage
k=1 (same positions)	4.177	65.1	all
k=2 (2-offset ensemble)	**4.141**	**62.9**	all

- -0.035 nats / -3.4% perplexity on a *strictly comparable, full-coverage* metric, at $0.91\times$ training FLOPs
- Per-offset: offset 0 = 4.145, offset 1 = 4.138 — **no off-distribution penalty at all**

Why is there no offset penalty? (a useful surprise)

The model was "only trained at offset 0" — yet offset 1 scores *equally well*. Explanation:

- Training sequences are packed by chunking a continuous token stream at arbitrary boundaries
- So the window grid's alignment *relative to the text* was effectively random throughout training
- RoPE geometry is identical across offsets (adjacent positions are always k raw tokens apart)

Consequences

- Offset-ensemble eval is retroactively valid for all existing checkpoints — no retraining needed
- Token-by-token generation is unlocked: rotate the offset so a window boundary always ends at the current position (k rotating KV caches $\Rightarrow$ amortized incremental decoding)

Pooling ablations: what changes inside each window?

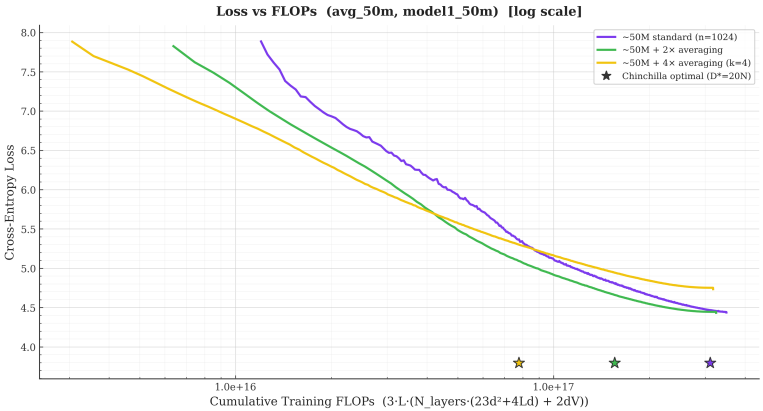
All runs use the same 50M OLM backbone and raw `seq_len`= 1024. Only the pooling operation changes.

Method	Construction	Question
Mean	equal fixed weights	Is the simplest rule sufficient?
Learnable (content)	content-scored learned weights	Can the model preserve useful tokens?
Learnable + position	content score + learned slot bias	Do content and position combine?
Exponential	fixed recency-biased weights	Is the latest token most useful?
Overlap (k=2)	window 4, stride 2	Do hard window boundaries hurt?
Word-aligned (k=2)	variable 2–4 token groups	Does splitting words hurt?

Budgets: k=2: 2.00B raw tokens; k=4: 4.07B raw tokens. Reported values are offset-0 eval CE (nats/token).

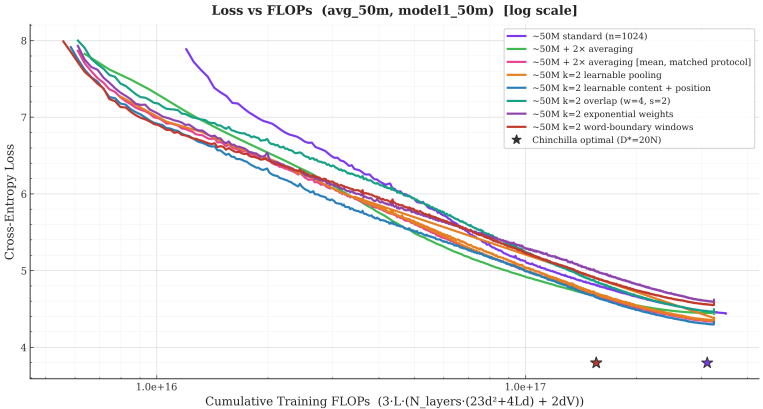
New since last time: mean pooling has been **rerun under the exact matched protocol** at both k (the old canonical mean runs used a different update schedule), and the **learnable + position** pooler has finished at both k.

Pooling ablations: loss vs training FLOPs



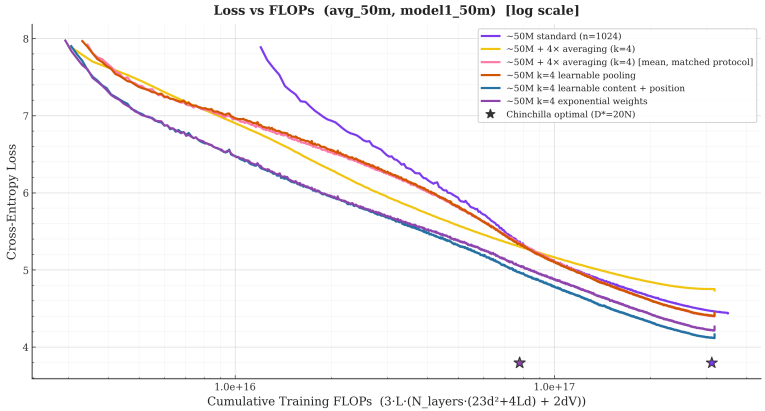
FLOPs are recomputed from raw tokens and transformer length; the CSV `cumulative_flops` column is ignored. Curves show offset-0 eval CE.

k=2 pooling ablations: loss vs training FLOPs



All k=2 variants at 2.00B raw tokens under the matched protocol (old-protocol mean shown for reference). Learnable + position and matched mean track each other closely; the gap opens late in training.

k=4 pooling ablations: loss vs training FLOPs



All k=4 variants at 4.07B raw tokens under the matched protocol (old-protocol mean shown for reference). The positional-prior methods (learnable + position, exponential) separate from the position-blind ones (mean, content-only learnable) early and stay ahead.

Pooling ablations: combined results (k=2 and k=4)

	k=2 (2.00B)		k=4 (4.07B)	
Pooling	Final CE	PPL	Final CE	PPL
Learnable + position	**4.326**	**75.7**	**4.163**	**64.3**
Mean (matched rerun)	4.363	78.5	4.446	85.2
Learnable (content only)	4.383	80.1	4.445	85.2
Exponential (fixed)	4.617	101.2	4.265	71.1
Overlap (w=4, s=2)	4.492	89.3	–	–
Word-aligned	4.577	97.2	–	–
Mean (old protocol, ref.)	*4.433*	*84.2*	*4.734*	*113.7*

One rule wins both regimes

Learnable + position is the only method that is best at *both* compression ratios. Every other rule is regime-specific: mean is second at k=2 but poor at k=4; exponential is second at k=4 but worst at k=2.

k=2 pooling ablations: detailed results (now fully matched)

Pooling	25%	50%	75%	Final	Final PPL
Learnable + position	5.170	4.619	4.389	**4.326**	75.7
Mean (matched rerun)	5.174	4.649	4.425	4.363	78.5
Learnable (content only)	5.222	4.675	4.443	4.383	80.1
Overlap (w=4, s=2)	5.460	4.820	4.562	4.492	89.3
Word-aligned	5.413	4.872	4.640	4.577	97.2
Exponential	5.439	4.962	4.707	4.617	101.2
Mean (old protocol, ref.)	*5.061*	*4.638*	*4.480*	*4.433*	*84.2*

- Learnable + position is the best rule: 0.037 nats ahead of matched mean at the endpoint, and ahead at every checkpoint
- Revision of last time's conclusion: with the matched mean control in place, **content-only learnable no longer beats mean** (4.383 vs 4.363) — its earlier lead was the update-schedule confound
- Matched mean (4.363) vs old-protocol mean (4.433): the protocol alone is worth 0.070 nats at k=2
- Recency bias, overlap, and word alignment all remain worse than plain mean under matched conditions

All six runs above the rule share the same token budget (2.00B) and training schedule; checkpoints at 25/50/75/100%.

k=4 pooling ablations: position matters, and content adds on top

Pooling	25%	50%	75%	Final	Final PPL
Learnable + position	4.951	4.463	4.224	**4.163**	64.3
Exponential (fixed)	5.041	4.569	4.330	4.265	71.1
Mean (matched rerun)	5.326	4.756	4.505	4.446	85.2
Learnable (content only)	5.315	4.753	4.512	4.445	85.2
Mean (old protocol, ref.)	*5.290*	*4.934*	*4.786*	*4.734*	*113.7*

- Learnable + position wins at every checkpoint: 0.102 nats below exponential at the endpoint (PPL 64.3 vs 71.1) — initialized at the exponential profile, then content adapts on top
- The recency prior is confirmed against a **matched** mean control now: exponential beats matched mean by 0.181 nats
- Content-only learnable lands exactly on matched mean (4.445 vs 4.446): a position-blind scorer adds *nothing* at k=4 — position was the missing ingredient all along
- Matched mean (4.446) vs old-protocol mean (4.734): the update schedule alone is worth 0.288 nats at k=4

Why does learned pooling lose to fixed weights at k=4?

The content-only learnable pooler is deliberately small:

$$s_i = w^\top e_i + b, \qquad \alpha_i = \mathrm{softmax}(s_i), \qquad \bar{e} = \sum_{i=1}^{k} \alpha_i e_i.$$

- The scorer sees each token embedding independently, *before* the transformer and before positional information is added
- It is therefore **content-aware but position-blind**: the same token receives the same score whether it is first or last in a window
- Exponential pooling supplies the missing inductive bias directly: later tokens always receive more weight
- At k=2, this prior is too aggressive; at k=4, avoiding equal dilution matters enough that the recency prior wins decisively

Mechanism ablation: done, and it confirms the story

We added a learned positional bias, $s_i = w^\top e_i + b + p_i$, with p_i initialized at uniform (k=2) / the exponential profile (k=4). Result: best rule at both k (4.326 at k=2; 4.163 at k=4). The content-only scorer ties matched mean at k=4 (4.445 vs 4.446), so position was the missing ingredient; content then adds a further 0.102 nats on top of the fixed prior. Next: inspect the learned α_i by position and token type.

Learnable + position pooling: definition

For a window of k embeddings $e_1, \ldots, e_k$, the pooler scores each slot with a shared content term plus a learned per-slot bias:

$$s_i = w^\top e_i + b + p_i, \qquad \alpha = \mathrm{softmax}(s_1, \ldots, s_k), \qquad \bar{e} = \sum_{i=1}^{k} \alpha_i \, e_i.$$

- **Parameters:** $w \in \mathbb{R}^d$, $b \in \mathbb{R}$, $p \in \mathbb{R}^k$ $\Rightarrow$ $d + 1 + k$ total (515 at k=2, 517 at k=4 for d=512) — negligible vs the 50M backbone
- **Initialization at the best fixed rule of each regime:** $w = 0$, $b = 0$ always; $p = 0$ at k=2 (starts exactly at *mean* pooling) and $p_i \propto$ ln-spaced exponential profile (last/first weight ratio 20) at k=4 (starts exactly at *exponential* pooling)
- **Decomposition by construction:** p_i carries the positional prior the content-only scorer cannot express ($s_i = w^\top e_i + b$ is position-blind); $w^\top e_i$ adds content adaptation on top
- Trained end-to-end with the ordinary LM loss; no auxiliary objective

What can we conclude from the pooling sweep?

Supported by fully matched runs

- Learnable + position is the best rule at both k (-0.037 vs mean at k=2; -0.283 at k=4)
- Content-only learnable does *not* beat matched mean (its earlier lead was the protocol confound)
- At k=4 the positional prior does the heavy lifting (-0.181 nats); content adds -0.102 more
- Overlap, word alignment, and pure recency all lose to plain mean at k=2
- Optimal pooling is k-**dependent**; the hybrid subsumes both regimes

Mechanism picture (now supported)

- Small k: both tokens carry signal; near-uniform weights plus mild content adaptation win
- Large k: the window's last token is the predictive one; an explicit positional prior preserves it, and a position-blind scorer cannot learn this
- Pooling controls which information survives before attention begins — worth 0.28 nats at k=4 for 517 parameters
- Protocol effect is large (0.07–0.29 nats): matched controls were essential

Pooling ablations: control status

Done: matched mean controls

Mean k=2 and k=4 have been rerun under the exact ablation protocol (batch, update count, warmup exposure, tied embeddings, eval batches). Every pooling comparison on the previous slides is now within one matched protocol. The protocol itself moves mean by 0.070 nats at k=2 and 0.288 at k=4, which retroactively explains why content-only learnable seemed to beat mean last time.

- **Still open — seeds:** single run per variant; 2–3 seeds for learnable+pos and matched mean would put error bars on the 0.037-nat k=2 gap (the k=4 gaps are large enough to be safe)
- **Still open — metric:** curves report offset-0 loss; full-position offset-ensemble NLL rescoring is queued
- **Still open — word method:** variable compression and word-start targets change the prediction distribution; needs bits-per-byte rescoring
- **Done — k=1 update-count control** at fixed FLOPs: the control beats every k=2 variant (4.142 vs 4.326 best) — see the update-count control slides

Data hygiene: clean the duplicated/malformed learnable-k=2 log before publication plotting (the two recorded endpoints agree: 4.379 vs 4.383).

Plan 1: scaling — 500M is in, and the decay reached zero

Question (revised): does averaging beat an *update-matched* baseline at any scale?

- Multipliers vs the batch-16 baselines: 1.58× (50M) → 1.44× (125M) → 1.32× (250M) → 500M: gap crosses zero on the native metric — shrinking, as projected
- At 50M the update-count control flips the result outright (4.142 vs 4.326 for the best k=2); the 500M result is independent evidence pointing the same direction
- **Still open, now higher priority:** the same k=1 half-batch control at 125M/250M/500M, and seed replicates at 500M to know whether the crossover is real or a single unlucky seed
- 1B is no longer an obvious next spend until we know whether 500M's near-zero gap is signal or noise

Deliverable unchanged: Chinchilla-style $L(C)$ fits per family, but on update-matched pairs.

Logistics

- k=2 consumes 2× raw data: 250M k=2 (10B) already exhausts FineWeb sample-10BT ⇒ ≥100BT slice before 500M/1B
- Standardize on offset-ensemble NLL as the reported metric

Plan 2: consequences of the Config B result

Config A becomes the paper's configuration. Scaling runs (250M/500M/1B) proceed with seq_len fixed, transformer length = seq/k.

Config B becomes an ablation, and a useful one: it shows the Config A gains are *not* a context effect — and the full-attention control shows the context failure is method-independent.

The context-for-context result is a keeper: same loss as full attention at 1.26× less training compute and half the inference KV cache. If long context matters for an application, averaging is the cheaper route — worth demonstrating on data where context actually helps.

Follow-ups to make the context conclusion airtight:

- Loss-vs-position curves (Config B k=2 and full-attention k=1): does loss improve at positions with >1024 tokens of available context?
- Document-length distribution of FineWeb (most docs $<$ 1024 tokens)
- Long-document data (PG19, code) and/or document-boundary attention masking — the setting where the 1.26× advantage would translate into a headline long-context result

Plan 3: remaining controls after the pooling sweep

Controls

- [done] Mean k=2/k=4 matched reruns
- [done] Learnable + position at both k (best rule at both)
- Full-position offset-ensemble NLL for every pooling rule
- 2–3 seeds for learnable+pos and matched mean (the k=2 gap is only 0.037 nats)
- [done] k=1 update-count control — control wins; repeat at 125M/250M
- Coarser-tokenizer baseline ($\sim 2\times$ mean token length), compared in bits-per-byte

Method follow-ups

- Inspect learnable+pos weights: learned α_i by position and token type; position-wise loss
- Promote learnable+pos to the scaling runs? (517 params, best at both k) — decide before 500M
- Add strided subsampling as an information-content control
- Clean $k \in \{2, 4, 8\}$ sweep under one training protocol
- Small local decoder to emit all k tokens (Block-Transformer style) $\Rightarrow$ native generation + downstream zero-shot evals (LAMBADA, HellaSwag, PIQA, ARC-E)

Positioning: closest prior work

Patch-Level Training for LLMs (Shao et al., 2024; ICLR'25) — nearly the same mechanism: average K token embeddings, predict the next patch, then *fine-tune at token level*. 370M–2.7B, claims 0.5× cost.

Our deltas (to sharpen):

- Patch-level as the *final operating point*, not a warm-up — made viable by the offset-ensemble eval + rotating-offset generation
- FLOP-controlled *scaling-law* treatment (efficiency vs scale), not single budget points
- Context-extension angle (Config B): same positions, double context, half the inference KV cost

Also related: MegaByte, Byte Latent Transformer (dynamic patches), Block Transformer, Funnel/Hourglass, multi-token prediction (Gloeckle et al.; DeepSeek-V3 MTP), context compression (gist tokens, AutoCompressor, ICAE).

Summary / discussion

- Vs batch-16 baselines, averaging wins at 50M/125M/250M: -3.4% perplexity at $0.91\times$ FLOPs on the full-coverage metric at 125M; equal-loss savings $1.58\times$ (50M), $1.44\times$ (125M), $1.32\times$ (250M) — but the multiplier shrinks with scale
- At 500M the gap crosses zero: native loss slightly *favors the baseline* (-0.003), full-position still barely favors averaging ($+0.003$) — both far below our 0.070-nat noise floor, and the same direction as the update-count control
- The 50M update-count control flips the headline: a $k{=}1$ baseline with matched updates and positions beats every $k{=}2$ variant (4.142 vs 4.326 best) at comparable FLOPs. Update schedule alone is worth -0.295 nats; the $k{=}2$ edge was -0.074
- Evaluation gap closed with offset-ensemble NLL — retroactively valid, and it unlocks generation
- Mechanism decomposed: the gain vs batch-16 baselines is **data-per-FLOP, not context** — extended context loses at 50M *by any method* ($+0.21$ nats for both $k{=}2$ averaging and $k{=}1$ full attention)
- Context-for-context: averaging matches full attention at 2048 context with $1.26\times$ less compute and half the inference KV cache
- Pooling sweep, now with matched mean controls: learnable + position is the best rule at both k (-0.037 nats vs mean at $k{=}2$, -0.283 at $k{=}4$); content-only learnable ties mean, so position is the key ingredient at $k{=}4$
- Remaining pooling caveats: single seed per variant, offset-0 metric (full-position rescoring queued)

Questions for discussion

Questions for discussion

- The 500M gap is within noise and we have one seed: is it worth 2–3 seed replicates at 500M before deciding whether to spend on 1B, or do we treat the crossover as decisive on its own?
- The update-count control at 50M reverses the headline: run the same control at 125M/250M/500M first, or also give k=2 the half-batch treatment to test whether small-batch gains are symmetric?
- If averaging loses update-matched everywhere: is context-for-context (same loss, half the KV cache) plus the pooling findings a sufficient paper, or does the project pivot?
- Promote learnable + position pooling to any remaining scaling runs, or keep the scaling story on parameter-free mean pooling?
- Is the context negative result worth a deeper treatment (loss-vs-position, long-document data) or one ablation slide?